

Sommaire

Projet Book'ing API — Documentation technique	1
1. Présentation.....	1
2. Objectifs	1
3. Stack technique.....	2
4. Structure du projet.....	2
5. Base de données	2
6. Fonctionnalités	3
7. Logique technique — points clés.....	6
8. Difficultés rencontrées.....	7
9. Compétences BTS SIO mises en œuvre.....	8
10. Bilan et améliorations possibles.....	8
11. Liens utiles	8

Projet Book'ing API — Documentation technique

Auteur : William TOUSSON · BTS SIO SLAM 1^{re} année · IPSSI SQY **Période :** 07/03/2025 – 15/03/2025 **Type :** TP individuel — Réalisation en cours de formation **Dépôt :** <https://github.com/william-tsn/book-api>

1. Présentation

Application web de **recherche et de gestion de livres favoris** alimentée par l'**API Google Books**. L'utilisateur s'inscrit, recherche un livre par titre, l'ajoute à ses favoris et peut gérer son profil personnel (pseudo, mot de passe, photo).

Le projet est une application **PHP / MySQL** classique exécutée localement via XAMPP, avec un front-end stylé en Tailwind CSS et une logique de recherche en JavaScript Vanilla.

2. Objectifs

- Mettre en pratique le **PHP côté serveur** (sessions, authentification, accès à une base de données).
- Concevoir une **base de données relationnelle** avec relation utilisateur ↔ favoris.
- Consommer une **API REST externe** (Google Books) depuis le navigateur.
- Pratiquer l'**authentification** : inscription, connexion, sessions PHP, déconnexion.
- Réaliser une interface cohérente et responsive avec **Tailwind CSS**.

3. Stack technique

Composant	Détail
HTML5	Pages PHP (home.php, login.php, inscription.php, profile.php...)
Tailwind CSS	Chargé via CDN (cdn.tailwindcss.com)
Font Awesome	Icônes (recherche, profil) via CDN
JavaScript Vanilla	scripts/app.js (logique de recherche et favoris)
PHP	Sessions, authentification, accès BDD
MySQL	2 tables : users et favoris
API externe	Google Books API
Environnement local	XAMPP (Apache + MySQL + PHP)

4. Structure du projet

book-api/	
├── index.php	← Redirige vers pages/home.php
├── style.css	← (présent dans le repo, actuellement vide)
├── database.sql	← Script SQL de création de la BDD
├── pages/	
│ ├── home.php	← Page d'accueil + barre de recherche
│ ├── login.php	← Connexion
│ ├── inscription.php	← Inscription
│ ├── profile.php	← Modification du profil
│ ├── logout.php	← Déconnexion
│ └── favorite.php	← Page des favoris (référéncée dans le menu)
├── component/	
│ └── header.php	← Navbar partagée
└── scripts/	
└── app.js	← Logique JS recherche + ajout favori

5. Base de données

Le projet utilise **deux tables** liées par une clé étrangère, créées par le script database.sql.

Table users

Colonne	Type	Contrainte
id	INT(11)	PK, AUTO_INCREMENT
pseudo	VARCHAR(128)	NOT NULL
nom	VARCHAR(50)	NOT NULL

Colonne	Type	Contrainte
prenom	VARCHAR(50)	NOT NULL
password	VARCHAR(255)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
photo_de_profil	VARCHAR(255)	nullable

Table favoris

Colonne	Type	Contrainte
id	INT(11)	PK, AUTO_INCREMENT
user_id	INT(11)	FK → users(id) ON DELETE CASCADE
book_id	VARCHAR(50)	NOT NULL (ID Google Books)
title	VARCHAR(255)	NOT NULL
author	TEXT	NOT NULL
image_url	TEXT	nullable

La contrainte ON DELETE CASCADE assure que **si un utilisateur est supprimé, tous ses favoris le sont automatiquement** — c’est une bonne pratique d’intégrité référentielle.

6. Fonctionnalités

6.1 Accueil — sans être connecté

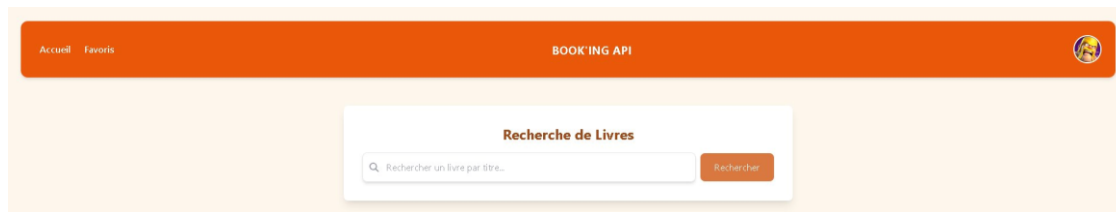
L'utilisateur qui n'est pas authentifié voit un message rouge l'informant qu'il doit se connecter pour accéder à la recherche.



Page d'accueil sans connexion

6.2 Accueil — connecté

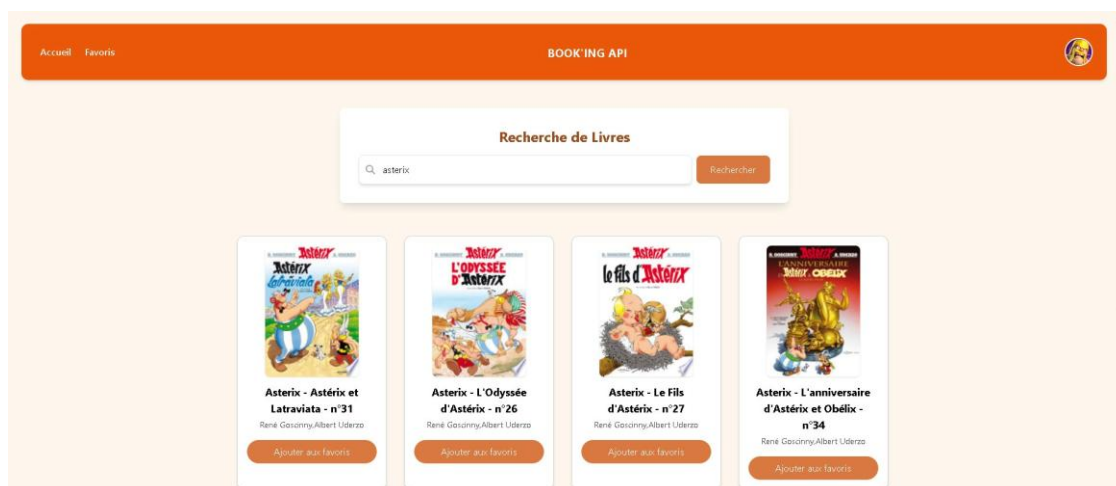
Une fois connecté, la barre de recherche apparaît avec un champ texte et un bouton « Rechercher ».



Page d'accueil connecté

6.3 Recherche de livres

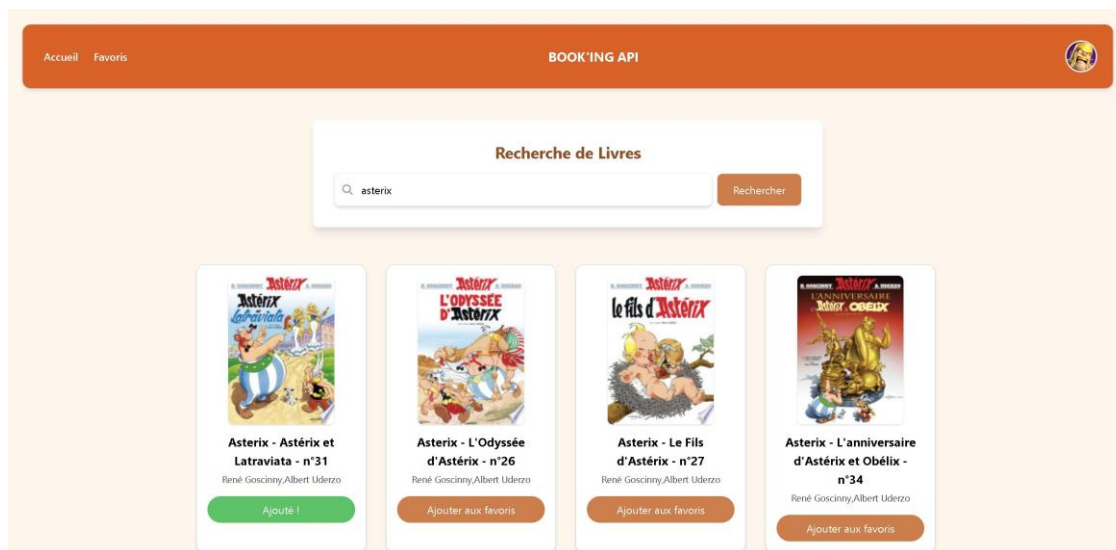
La saisie d'un titre déclenche un appel à l'API Google Books. Les résultats sont affichés sous forme de **grille responsive de cartes** (1, 2, 3 ou 4 colonnes selon la largeur d'écran), chacune contenant la couverture, le titre, le ou les auteurs, et un bouton « Ajouter aux favoris ».



Résultats de recherche

6.4 Ajout aux favoris

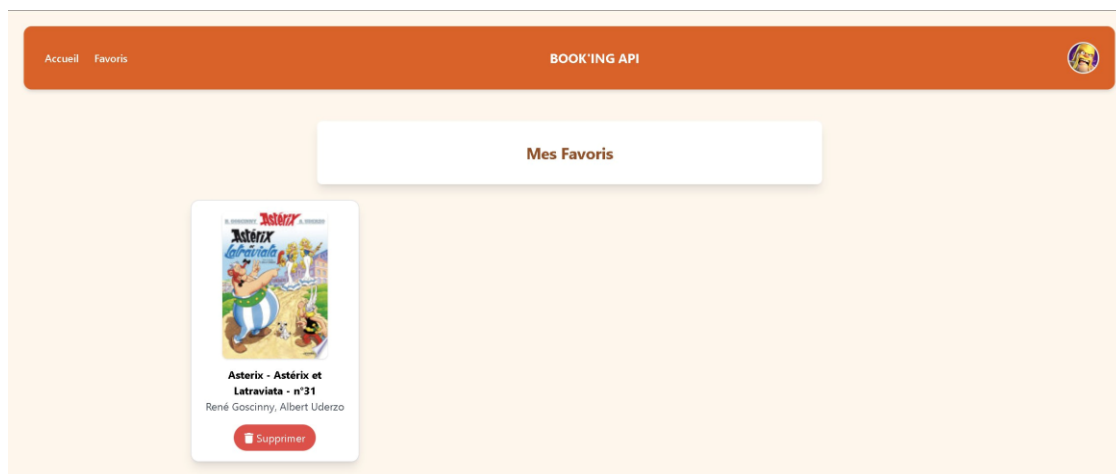
Le clic sur « Ajouter aux favoris » envoie une requête POST vers `config/favoris.php` avec les infos du livre. En cas de succès, le bouton passe en **vert avec le texte « Ajouté ! »** et devient désactivé pour empêcher les doublons.



Bouton après ajout aux favoris

6.5 Page Favoris

Accessible depuis le menu, la page « Mes Favoris » liste tous les livres ajoutés par l'utilisateur connecté. Chaque carte affiche le livre avec un bouton « **Supprimer** » rouge.



Page Mes Favoris

6.6 Modification du profil

L'utilisateur peut éditer trois informations indépendamment : son pseudo, son mot de passe et l'URL de sa photo de profil. Chaque champ a son propre bouton « Appliquer » et un bouton global « Sauvegarder les changements ».

Accueil Favoris BOOK'ING API

Modifier votre profil

Pseudo Appliquer

Mot de passe Appliquer

URL de la photo de profil Appliquer

Sauvegarder les changements

Page profil vide

Une fois les champs remplis, l'aperçu du nouveau pseudo et de la photo est visible.

BOOK'ING API Accueil Favoris

Modifier votre profil

 Aperçu du nouveau pseudo et photo

Nom d'utilisateur Appliquer

Mot de passe Appliquer

URL de la photo de profil Appliquer

Sauvegarder les changements

Page profil rempli

7. Logique technique — points clés

7.1 Appel à l'API Google Books

```
fetch(`https://www.googleapis.com/books/v1/volumes?q=intitle:${input.value}&key=${apiKey}`)  
  .then(res => res.json())
```

```
.then(data => displayBook(data.items))
.catch(error => console.log(error));
```

Le paramètre `intitle:` filtre la recherche par titre exclusivement. L'API renvoie un tableau d'objets `items`, chacun contenant un sous-objet `volumeInfo` (titre, auteurs, miniature, etc.).

7.2 Ajout d'un favori (POST asynchrone)

```
fetch("../config/favoris.php", {
  method: "POST",
  headers: { "Content-Type": "application/x-www-form-urlencoded" },
  body: new URLSearchParams(bookData)
})
.then(response => response.json())
.then(data => {
  if (data.success) {
    favBtn.textContent = "Ajouté !";
    favBtn.disabled = true;
    favBtn.classList.replace("bg-[#d97941]", "bg-green-500");
  }
});
```

Cette logique combine **JavaScript (fetch)** et **PHP côté serveur**, démontrant la communication entre le client et le serveur sans rechargement de page.

7.3 Gestion de session PHP

Chaque page PHP commence par `session_start()`. La variable `$_SESSION['user_id']` détermine l'affichage de la barre de recherche (cf. capture 6.1 vs 6.2) :

```
<?php if (isset($_SESSION['user_id'])): ?>
  <!-- Barre de recherche -->
<?php else: ?>
  <p>Vous devez être connecté pour rechercher des livres.</p>
<?php endif; ?>
```

8. Difficultés rencontrées

- *Installation et configuration de XAMPP, démarrage Apache et MySQL.*
 - *Création de la BDD et import du script SQL via phpMyAdmin.*
 - *Compréhension du fonctionnement des **sessions PHP** pour distinguer utilisateur connecté / non connecté.*
 - *Premier projet avec une **architecture client-serveur** : comprendre que le JS et le PHP communiquent par fetch + JSON.*
 - *Gestion de la **photo de profil par URL externe** (vs. upload de fichier).*
 - *Différencier les **endpoints** Google Books API (`intitle:`, `inauthor:`, etc.).*
-

9. Compétences BTS SIO mises en œuvre

Ce projet justifie les compétences suivantes du **Bloc 1** :

- **C3 — Développer la présence en ligne de l'organisation** → Sous-compétence : « *Participer à l'évolution d'un site Web exploitant les données de l'organisation* ». Le site enrichit son interface avec les données distantes de Google Books et stocke localement les favoris des utilisateurs.
 - **C6 — Organiser son développement professionnel** → Sous-compétence : « *Mettre en place son environnement d'apprentissage personnel* ». Le projet a permis l'auto-apprentissage de PHP côté serveur, de MySQL et du concept d'authentification par session.
-

10. Bilan et améliorations possibles

Ce que le projet m'a apporté :

- Compréhension de l'**architecture client-serveur** (front HTML/JS ↔ back PHP/MySQL).
- Pratique de l'**authentification** : inscription, hash de mot de passe, sessions.
- Première **conception de base de données** avec une clé étrangère et une suppression en cascade.
- Communication entre une page web et une **API REST tierce** (Google Books).
- Manipulation de Tailwind CSS pour produire une interface responsive.

Pistes d'amélioration identifiées :

- **Sécurité : ne pas exposer la clé API Google Books côté client.** Actuellement la clé est en clair dans `scripts/app.js`, donc visible par n'importe quel visiteur. Solution : faire transiter les requêtes Google Books **par le PHP** (proxy serveur) plutôt que directement depuis le navigateur.
 - **Validation et messages d'erreur utilisateur** : remplacer les `alert()` et `console.log` par des messages affichés dans l'interface.
 - **Pagination des résultats** : actuellement seuls les premiers résultats de l'API sont affichés.
 - **Recherche multi-critères** : auteur, ISBN, éditeur (via les autres paramètres Google Books).
 - **Tailwind en production** : remplacer le CDN par une installation locale (le CDN de Tailwind est destiné au prototypage).
 - **Compléter le fichier `style.css`** actuellement vide dans le repo, ou le supprimer.
-

11. Liens utiles

- Dépôt GitHub du projet : <https://github.com/william-tsn/book-api>

- Documentation Google Books API :
<https://developers.google.com/books/docs/v1/using>
- Documentation Tailwind CSS : <https://tailwindcss.com/docs>
- Documentation PHP — Sessions :
<https://www.php.net/manual/fr/book.session.php>